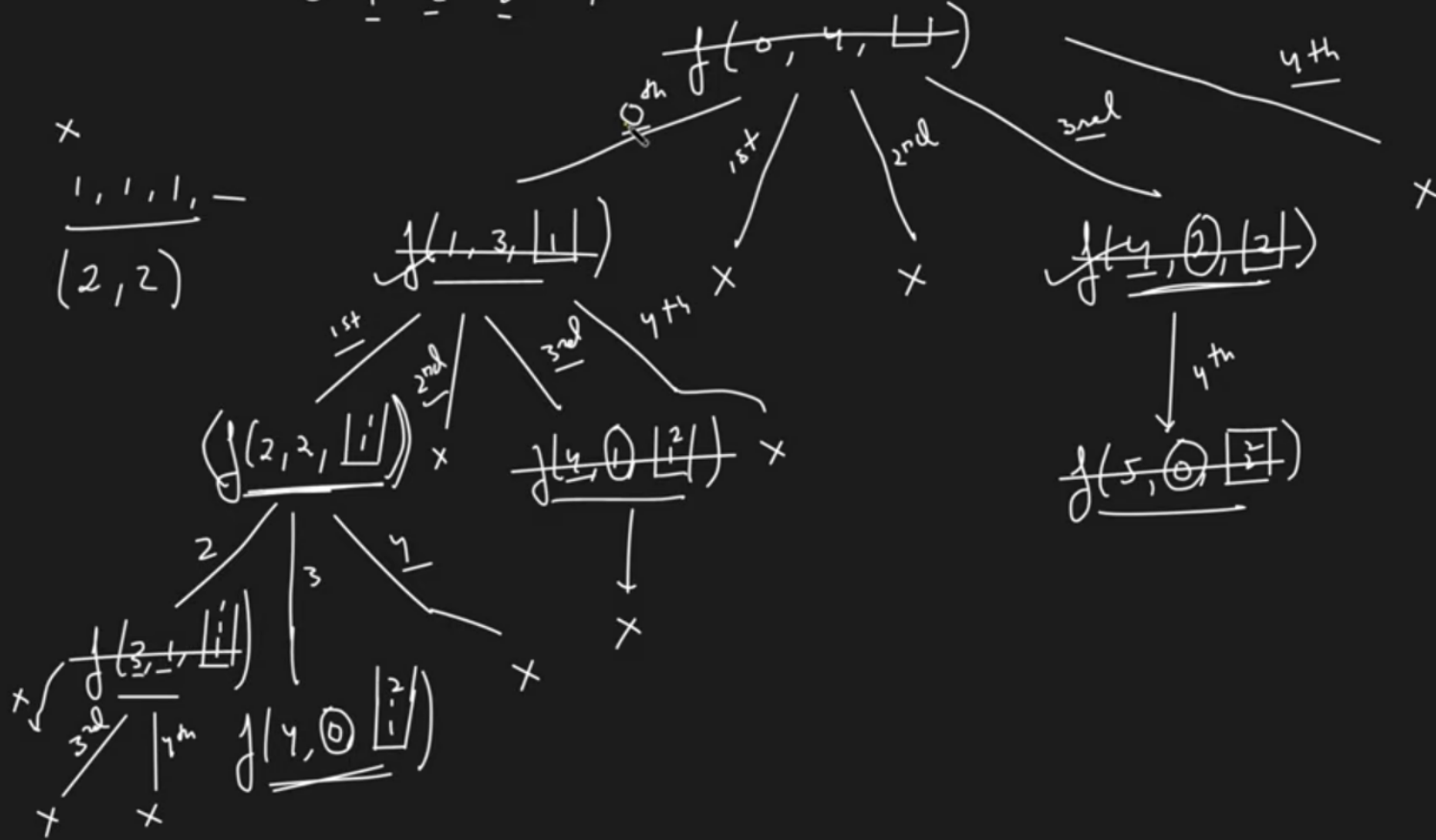


$$\text{arr} = \begin{bmatrix} 1 & 1 & 1 & 2 & 2 \\ 0 & 1 & 2 & 3 & 4 \end{bmatrix}$$

$$\text{target} = 4$$

$$\begin{bmatrix} (1, 1, 2) \\ (2, 2) \end{bmatrix}$$



target = 4

$$\leftarrow \left[\frac{(1, 1, 2)}{(2, 2)} \right] \rightarrow \text{arg}$$

yes (+ weight = 0)
ans. add (ds)

$$\begin{array}{l} TC \rightarrow 2^n \times k \\ \hline SC \rightarrow k \times n \end{array}$$
$$f(\text{ind}, \text{target}, \text{ds}, \text{ans})$$

loop (ind $\rightarrow n-1$) $\rightarrow$ (i)

```
if (arr[i] > +arr[i+1])
    break;
```

Q3. add (arr[i])

$$f_{i+1}, \text{ length} = \text{arr}[i], \text{ds}, \text{ans}$$
~~ds. remove (-)~~

```
1 class Solution {
2     public:
3     void findCombination(int ind, int target, vector<int> &arr, vector<vector<int>> &ans, vector<int>&ds) {
4         if(target==0) {
5             ans.push_back(ds);
6             return;
7         }
8         for(int i = ind; i<arr.size(); i++) {
9             if(i>ind && arr[i]==arr[i-1]) continue;
10            if(arr[i]>target) break;
11            ds.push_back(arr[i]);
12            findCombination(i+1, target - arr[i], arr, ans, ds);
13            ds.pop_back();
14        }
15    }
16    public:
17    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
18        sort(candidates.begin(), candidates.end());
19        vector<vector<int>> ans;
20        vector<int> ds;
21        findCombination(0, target, candidates, ans, ds);
22        return ans;
23    }
24};
```